Assignment 5: Inter-Process Communication

Operating System Lab (Unix)

Problem Statement

Write C programs to demonstrate various Inter-Process Communication (IPC) mechanisms in Unix/Linux. The objective is to understand how processes exchange data and synchronize execution using different IPC techniques.

Tasks to Implement

1. Write a program to demonstrate communication between parent and child process using `pipe()`
2. Write a program to demonstrate two-way communication using pipes
3. Write a program using named pipes (FIFO) for communication between two processes
4. Write a program to send and receive messages using message queues (`msgget()`, `msgsnd()`, `msgrcv()`)
5. Write a program to demonstrate shared memory using `shmget()`, `shmat()`, `shmdt()`
6. Write a program to synchronize processes using semaphores (`semget()`, `semop()`, `semctl()`)
7. Write a program to implement producer-consumer problem using shared memory and semaphores

Instructions

- Use C programming language
- Compile using `gcc filename.c -o output`
- Execute using `./output`
- Use headers: `unistd.h`, `sys/types.h`, `sys/ipc.h`, `sys/msg.h`, `sys/shm.h`, `sys/sem.h`
- Ensure proper cleanup of IPC resources after execution
- Use multiple terminals where required (FIFO, message queue)

Sample Output

Example (pipe):

```
Parent writing message...
Child received: Hello from parent
```

Example (shared memory):

```
Process 1 writing to shared memory...
Process 2 reading: Data received successfully
```

Constraints

- Programs must demonstrate actual IPC behavior
- Ensure synchronization where required
- Avoid race conditions in shared memory
- Properly remove IPC resources after execution

Evaluation Criteria

- Correct Implementation of IPC Mechanisms – 40%
- Understanding of Concepts – 20%
- Synchronization Handling – 15%
- Code Structure and Clarity – 15%
- Documentation / Comments – 10%

Submission Guidelines

- Submit all programs in a single compressed (.zip) file
- Folder name: `RollNo_Assignment5`
- Include source files and sample outputs
- Include a README explaining each IPC mechanism

Bonus Tasks (Optional but Recommended)

1. Implement chat between two processes using FIFO
2. Compare performance of pipes vs shared memory
3. Implement multi-producer multi-consumer problem
4. Demonstrate deadlock scenario using semaphores
5. Build a mini message broker using message queues